Day1 실습 서브노트
범용 모델을 회사 업무에 붙이려면 무엇이 부족한가

과목: sLLM 구현 및 Fine Tuning / 1일차
실습 모델: Qwen2.5-1.5B-Instruct, BGE-M3, FAISS, PEFT(LoRA)
돌려본 코드: knowledge_asset_demo_faiss_bge_m3, peft_lora_parameter_compare_cache,
scripts/train_lora.py (설정만 확인, 학습은 다음 실습)
실행 환경: Apple Silicon Mac, Qwen은 MPS FP16, Embedding은 CPU

오늘 한 일은 Fine-Tuning 자체가 아니라, Fine-Tuning을 하기 전에 "왜 해야 하는지"를 코드로 확인하는 쪽이었다. 노트북 두 개를 그대로 돌리고, 거기서 나온 숫자가 가이드에서 읽은 이야기와 맞는지 다시 세어봤다. 아래는 과제에 적힌 다섯 항목 순서대로, 내가 이해한 것과 실제로 본 결과를 같이 적은 것이다.

---

1. 이해관계자 가치 (Pain Point)

가이드에서 사전학습 모델을 "인터넷을 다 읽고 자란 신입사원"이라고 했는데, 돌려보니 그 비유가 과장이 아니었다. Qwen2.5-1.5B-Instruct는 한국어로 그럴듯한 문장을 잘 만든다. 그런데 실습에서 쓰는 가상 회사 Human AI Corporation의 규정은 원래 모른다.

예를 들면 이런 것들이다.

- AI 교육비 제도 코드가 HAI-EDU-2026이고, 1인당 연간 한도가 120만원이라는 사실
- 신청은 교육 시작 14일 전까지, 승인은 팀장 → HR 검토 → 최종 승인 순서라는 절차
- 수료율 80% 미만이면 지원 대상에서 빠진다는 예외
- 본격 SFT 데이터에 나오는 AURORA 재택 등급, BLUE-7 보안등급, ORBIT 긴급근무 코드처럼 회사 밖에선 의미가 없는 고유 용어

이게 그냥 "몰라서 답변을 못 한다"로 끝나면 차라리 낫다. 실제로는 더 골치가 아프다. 모델은 모르는 내용을 빈칸으로 두지 않고, 일반적인 기업 관행을 끌어와서 답을 만들어낸다. "교육비는 보통 연 얼마까지 지원됩니다", "재택은 주 2~3회가 일반적입니다" 같은 식이다. 직원 입장에서는 문장이 자연스러우니까 그게 우리 회사 규정인 줄 알고 넘어가기 쉽다.

가이드1에서 말한 두 가지 간극이 여기서 갈린다.

1. 지식의 간극: 우리 회사 규정·코드·한도를 모른다.

2. 말하는 방식의 간극: 설령 비슷한 내용을 알더라도, 근거를 대고 / 절차 순서로 말하고 / 없는 건 없다고 하는 회사 스타일로 답하지 못한다.

앞쪽은 Knowledge를 검색해서 붙여주는 방식으로 당장 메울 수 있고, 뒤쪽은 SFT로 모델 습관 자체를 바꿔야 한다. 오늘 실습이 전자를 직접 해보고, 후자의 입구(LoRA 구조)만 열어본 날이다.

누구의 불편이고, 특화가 되면 뭐가 달라지나

이해관계자	지금 겪는 일	도메인에 맞춘 sLLM이 줄 수 있는 것
현업 직원	연차·재택·교육비를 매번 담당자에게 물어본다. 사람마다 답이 조금 다르다.	사내 규정 기준으로 같은 답을 바로 받는다. 고유 코드(AURORA, BLUE-7 같은)도 빠지지 않는다.
HR / 총무	같은 질문을 반복해서 받는다. 예외 케이스까지 섞여 들어오면 응대가 밀린다.	문서에 있는 질문은 모델이 받고, 규정에 없는 질문만 사람이 본다.
보안 / 법무	사내 매뉴얼을 외부 상용 LLM에 넣기 부담스럽다. 계약상 "학습 안 함"이어도 검증이 어렵다.	1.5B 정도면 사내 장비에서 돌릴 수 있다. 데이터가 밖으로 안 나간다.
인프라 / 운영	큰 모델을 매번 호출하면 비용이 쌓인다. 전체 파라미터를 다시 학습하는 건 더 비싸다.	작은 모델 + LoRA 어댑터만 학습·저장하면 된다. 오늘은 그 규모가 어느 정도인지 숫자로 봤다.

교재에서 말한 내부 파이프라인의 두 목적 — 데이터 주권, 도메인 버티컬 — 이 이해관계자 표와 같은 이야기다. 중요한 건 모델이 작아서가 아니라, 회사 Knowledge Asset을 검색이든 학습이든 우리 쪽에 붙일 수 있다는 점이다.

---

2. Knowledge Asset과 sLLM 활용

기업이 가진 사내 문서, 규정, 매뉴얼은 모델 안에 들어 있지 않다. 오늘 실습은 그 문서를 모델 밖에 두고, 질문과 가까운 것만 찾아 Qwen에게 넘겨주는 방식이었다.

노트북에서 한 순서는 이렇다.

1. Knowledge 6건을 id / title / content로 정의했다. 교육비 4건, 리프레시 휴가, 재택근무.
2. 각 문서의 title + content를 BGE-M3로 Embedding했다. 문서당 1024차원 벡터가 나왔고, `normalize_embeddings=True`라서 벡터 길이가 1이다. 그래서 내적이 곧 코사인 유사도처럼 동작한다.
3. 그 6개 벡터를 FAISS `IndexFlatIP`에 넣었다. 문서가 적어서 근사 검색이 아니라 전수 비교다.
4. 질문이 들어오면 같은 BGE-M3로 질문을 Embedding하고, 가까운 문서 2건(top_k=2)을 꺼낸다.

5. 꺼낸 2건만 System Prompt의 [Retrieved Knowledge]에 넣고, "여기 적힌 것만 근거로 답하고 없으면 없다고 하라"고 Qwen에게 지시한다.

직접 돌려서 나온 검색·답변이다. 질문 문장이 문서와 똑같지 않아도 의미로 맞는 쪽을 찾아왔다.

질문	Top-1 (score)	Top-2 (score)	Qwen 답변
AI 교육비는 연간 얼마까지 지원받을 수 있나요?	HR-EDU-001 지원 한도 (0.7812)	HR-EDU-004 제외 조건 (0.6470)	연간 최대 120만원. 맞다.
AI 교육비 신청은 교육 시작 며칠 전까지 해야 하나요?	HR-EDU-002 신청 기한 (0.8096)	HR-EDU-003 승인 절차 (0.6302)	최소 14일 전. 맞다.
재택근무는 일주일에 몇 번 가능한가요?	HR-WORK-001 재택근무 (0.7673)	HR-LEAVE-001 리프레시 휴가 (0.4619)	주 최대 2회. 맞다.
해외 AI 컨퍼런스 항공료를 전액 지원하나요?	HR-EDU-001 지원 한도 (0.5647)	HR-EDU-003 승인 절차 (0.4470)	"현재 Knowledge Base에서 해당 내용을 확인할 수 없습니다." 거절했다.

여기서 확인한 것은 이것이다. Base Qwen이 회사 규정을 몰라도, 관련 문서를 찾아서 Context로 넣어주면 그 범위 안에서는 답한다. 마지막 질문처럼 Knowledge에 없는 내용은 FAISS가 그래도 "제일 가까운 문서"를 내준다. score가 0.56 정도로 낮다. 그래도 System Prompt가 "없으면 없다고 하라"고 못 박혀 있어서, 이번엔 항공료를 지어내지 않고 거절했다.

다만 검색이 맞았다고 답도 맞는 건 아니었다. "수료율이 70%라면 교육비 지원을 받을 수 있나요?"에서 FAISS는 정답 문서(HR-EDU-004, 80% 미만 제외)를 Top-1(0.6913)으로 잘 찾았다. 그런데 Qwen 답은 이랬다.

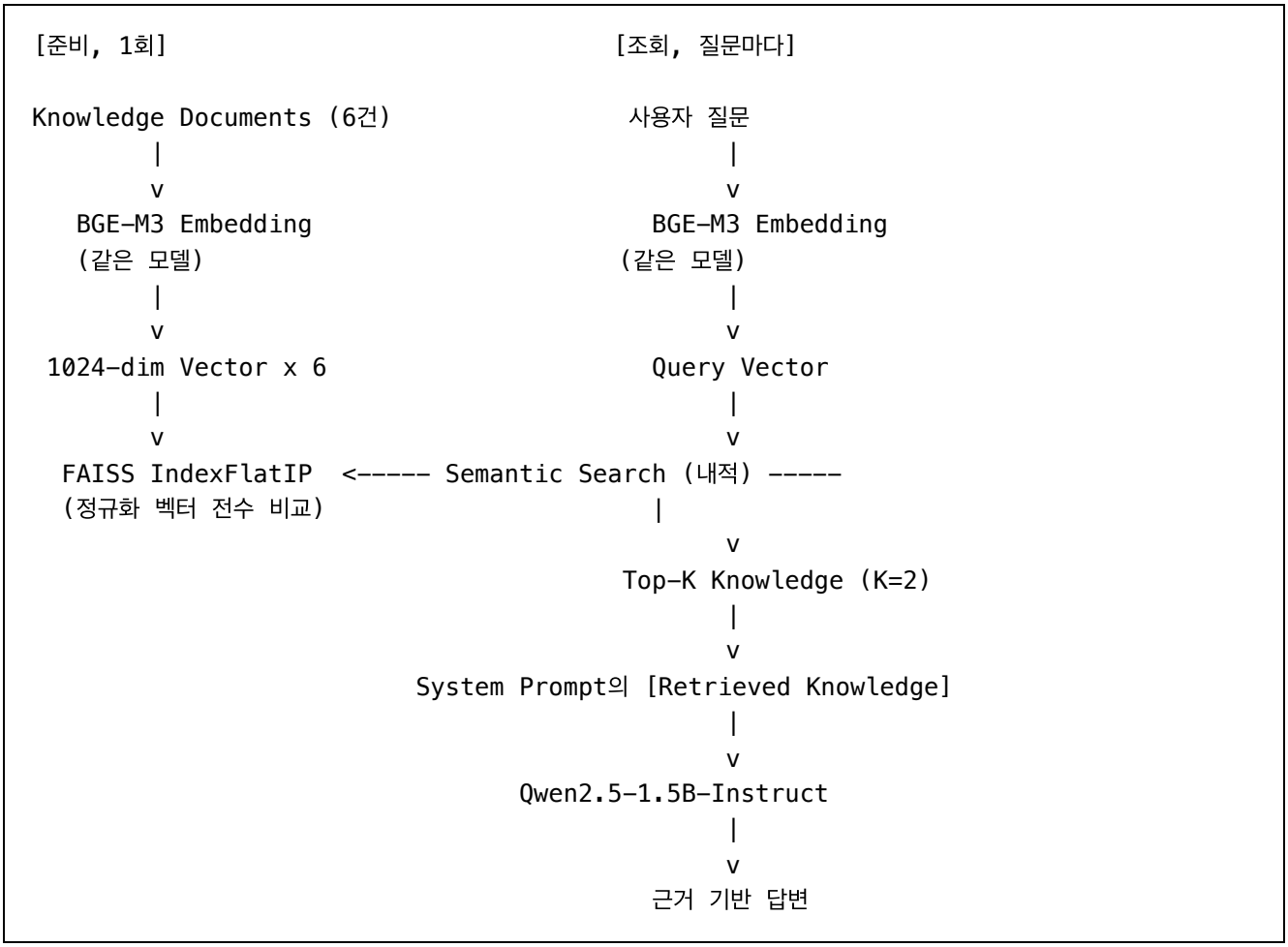
> 네, 수료율이 70%라면 교육비 지원을 받을 수 있습니다. 그러나 이 경우 지원금액은 120만원보다 적게 지급될 것입니다.

문서에는 "80% 미만이면 제외"라고만 적혀 있다. 70은 80보다 작으니 제외가 맞는데, 모델이 산수를 놓쳤다. 게다가 top_k=2로 같이 들어온 HR-EDU-001(120만원 한도)을 섞어서, 없는 규정("적게 지급")까지 만들어냈다. 검색은 모델 밖에 지식을 붙이는 방법이지, 작은 생성 모델의 조건 판단을 보증하지는 않는다. 이 지점이 RAG만으로 끝내기 어려운 이유고, 다음 실습의 SFT가 필요한 이유이기도 하다.

---

3. Mini Knowledge Base 구성도

흐름은 두 갈래다. 왼쪽은 문서를 인덱스로 만드는 준비 단계(한 번), 오른쪽은 질문이 들어올 때마다 반복되는 조회 단계다.



구조에서 내가 붙잡아 둔 점.

- 문서 Embedding과 질문 Embedding은 반드시 같은 모델이어야 한다. 공간이 달라지면 거리를 비교할 수 없다.
- Qwen에는 문서 6건을 다 넣지 않는다. 검색된 2건만 넣는다. 문서가 나중에 수천 건이 되어도 프롬프트가 같이 커지지 않는 이유다.
- IndexFlatIP는 근사 검색이 아니다. 벡터를 정규화해 두면 전수 내적 = 코사인 유사도다. 문서 6건, 차원 1024면 인덱스는 약 24KB다. (문서 1건당 4KB)
- FAISS는 "가장 가까운 것"을 항상 돌려준다. 관련 없는 질문에도 점수가 나온다. 그래서 유사도 Threshold 같은 장치가 따로 필요하다. 이건 부록에서 실제로 나눠봤다.

---

4. PEFT(LoRA) 적용 전·후 Parameter 비교

`peft_lora_parameter_compare` 노트북을 그대로 돌렸다. 설정은 가이드 워밍업과 같다. r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"], lora_dropout=0.05, bias="none".

모델을 로드하고 config도 찍어봤다. Qwen2.5-1.5B-Instruct는 decoder-only(CLM)다.

```
model_type          : qwen2
is_encoder_decoder  : False
hidden_size         : 1536
num_hidden_layers   : 28
num_attention_heads : 12
num_key_value_heads : 2
intermediate_size   : 8960
```

구분	전체 Parameter	학습 가능 (Trainable)	학습 가능 비율
LoRA 적용 전 (Base)	1,543,714,304	1,543,714,304	100.0000%
LoRA 적용 후 (q_proj, v_proj / r=8)	1,544,803,840	1,089,536	0.0705%

처음엔 "15억 개 중에서 109만 개를 고르는 것"이라고 생각했다. 가이드3 Q1이 그 오해를 먼저 짚는데, 숫자를 보니 이해가 됐다. 전체 파라미터는 거의 그대로다. 15.4억에서 약 109만 개가 늘었을 뿐이다. 늘어난 그 109만이 새로 붙인 LoRA 어댑터고, 학습되는 건 그것뿐이다. 원래 15.4억 개는 Freeze(`requires_grad=False`)된다. 그래서 학습 대상이 100%에서 0.07%로 줄었다.

109만 개가 어디서 나오는지

`requires_grad=True`인 이름을 출력해보면, 28개 레이어마다 q_proj / v_proj에 lora_A, lora_B가 붙어 있다. 레이어 하나당:

```
q_proj  lora_A [8, 1536] = 12,288    lora_B [1536, 8] = 12,288    → 24,576
v_proj  lora_A [8, 1536] = 12,288    lora_B [ 256, 8] =  2,048    → 14,336
                                                       레이어 합계 = 38,912

38,912 × 28 layers = 1,089,536
```

표의 Trainable 수와 같다. 여기서 의외였던 건 v_proj의 lora_B가 [1536, 8]이 아니라 [256, 8]인 점이다. config에 num_key_value_heads=2라고 나와 있다. GQA(Grouped-Query Attention)라서 Value 쪽 출력 차원이 1536이 아니라 256(= 2 × 128)이다. LoRA의 A, B 크기는 사람이 임의로 정하는 게 아니라, 붙는 Linear 층(W)의 모양을 따라간다. 가이드3 Q4, Q7이 코드로 보이는 지점이다.

그리고 q_proj라고 적으면 q_proj 하나만이 아니라, 28개 층 전부에 각각 다른 A/B가 붙는다. 아파트 각 층 현관에 스마트락을 따로 다는 비유 그대로였다.

왜 이 방식이 Fine-Tuning으로 효율적인가

- 메모리: Freeze된 15.4억 개는 옵티마이저 상태(모멘텀 등)를 안 들고 있어도 된다. 학습이 109만 개 기준이다.
- 저장: 결과물이 전체 모델이 아니라 어댑터 파일이다. 워밍업 설정 기준 FP32로 대략 4.16MB. 용도별로 여러 개를 만들어 갈아 끼울 수 있다.
- Catastrophic Forgetting: 원본 가중치를 안 고치니, 이미 가진 일반 언어 능력을 통째로 지울 위험이 작다. 책 원문은 두고 포스트잇만 붙이는 쪽에 가깝다.
- 저랭크 가설: 파인튜닝에 필요한 변화량(ΔW)은 원본만큼 복잡하지 않고, 몇 개 방향으로 표현된다는 것. r=8은 "그 방향을 최대 8개까지만 쓰겠다"는 상한이다.

한 가지 더. LoRA를 `get_peft_model`로 붙이자마자 Trainable 숫자가 바로 줄어 보인다. 가이드3 Q5대로, 이 순간에 정해지는 건 구조(종이 크기, 슬롯 개수)다. 그 안의 값이 의미 있게 채워지는 건 forward → loss → backward → optimizer step을 도는 학습 단계다. 오늘은 구조만 만들고, 값 학습은 하지 않았다.

---

5. SFT 적용 방향, 그리고 다음 실습과의 연결

SFT는 (질문, 정답) 쌍을 모범답안으로 주고 따라 쓰게 하는 것이다. 가이드1 비유로는 글쓰기 과외다. 배포된 데이터를 열어보면 형식이 이미 정해져 있다.

```
system  : 당신은 Human AI Corporation의 인사 규정 안내 도우미입니다.
          (근거, 고유 코드, 절차, 예외, 없으면 추측하지 말 것)
user    : AURORA 재택근무 등급에 대한 규정을 설명해 주세요.
assistant: 결론: ...
          적용 규정: AURORA-A / B / C / X ...
          위 규정만으로 판단하기 어려운 개별 상황은 인사팀에 확인해야 합니다.
```

모델이 배우는 관계는 "이 지시 + 이 질문 → 이런 형식·내용의 답"이다. 회사 Knowledge Asset을 이런 쌍으로 만들면, 검색 없이 짧은 질문만 들어와도 회사 말투와 고유 코드를 쓰게 만들 수 있다. 데이터 건수는 train 582, validation 97, evaluation 20이다. evaluation은 학습에 안 쓰고, 나중에 Base와 Fine-tuned를 같은 질문으로 비교하는 용도다.

오늘 한 검색과 SFT의 차이

	Knowledge 검색 (오늘)	**SFT / LoRA (다음 실습)**
지식이 있는 곳	모델 밖. 질문 시점에 찾아서 프롬프트에 넣음	모델 안. 어댑터 가중치에 학습으로 새김
바꾸는 것	모델은 그대로. 넣어주는 Context만 바뀜	모델의 답변 습관·형식이 바뀜

	Knowledge 검색 (오늘)	SFT / LoRA (다음 실습)
규정이 바뀌면	문서만 갈아끼우고 다시 Embedding	데이터를 고쳐서 재학습
잘하는 것	사실을 출처와 함께 물어다 줌. 최신성	짧은 질문에도 회사 형식·코드·거절 규칙을 지킴
약한 것	검색이 틀리면 답도 틀림. 작은 모델은 조건 판단에서 흔들림	새로운 방대한 사실 주입에는 부적합 (그건 CPT 영역). 재학습 비용

둘은 서로 대체재가 아니다. 교재에서도 RAG와 Fine-Tuning을 층이 다른 작업으로 나눈다. 실무에서는 SFT로 "어떻게 답할지"를 고정하고, 검색으로 "지금 근거가 되는 문장"을 붙이는 식으로 같이 쓴다. 오늘 70% 수료율 질문이 그 필요를 보여줬다. 문서는 맞았는데 모델이 조건을 잘못 읽었다. 그런 습관은 검색만으로 안 잡힌다.

다음 실습에서 할 일

- `hr_sft_train.jsonl`, `hr_sft_validation.jsonl`로 Qwen2.5-1.5B에 LoRA SFT를 실제로 돌린다. 수업 권장 설정은 MAX_LENGTH=384, EPOCHS=2, LR=1e-4.
- `scripts/train_lora.py`를 읽어보면 워밍업보다 범위가 넓다. q/k/v/o + gate/up/down, r=16. 오늘은 그 설정으로 Trainable이 약 1,846만 개(1.18%)까지 늘어난다는 것만 확인했다. 학습은 하지 않았다.
- 학습 중 loss와 eval_loss를 같이 본다. 결과물은 전체 모델이 아니라 `models/hr-qwen-lora/`의 어댑터다. Base Qwen + Adapter = Fine-tuned Qwen.
- `hr_eval.jsonl`로 같은 질문을 Base와 Fine-tuned에 넣고, Keyword Score / Token F1 / Hallucination을 비교한다. "SFT가 의도한 방향으로 행동을 바꿨는지"를 그때 숫자로 본다.

오늘 4번에서 본 것은 "학습될 상자가 얼마나 작은가"였다. 다음 실습은 그 상자 안에 회사 규정 답변 습관이 채워졌을 때, 실제로 답이 달라지는지를 보는 단계다.

부록. 가이드 밖에서 더 돌려본 것

A. 단어가 안 겹치는 질문으로 Semantic Search가 되는지

노트북 질문은 문서 제목이랑 단어가 많이 겹친다. "재택근무는 일주일에 몇 번"이면 검색이 너무 쉽다. 그래서 문서에 거의 없는 말로 바꿔서 다시 검색해 봤다.

질문	기대한 문서	실제 Top-1	score	결과
집에서 일해도 괜찮은가요?	HR-WORK-001 재택근무	HR-WORK-001	0.5460	맞음
오래 다닌 직원한테 주는 특별한 쉬는 날이 있나요?	HR-LEAVE-001 리프레시 휴가	HR-LEAVE-001	0.5810	맞음
HAI-EDU-2026 (코드만)	HR-EDU-001 지원 한도	HR-EDU-001	0.5308	맞음
강의를 얼마나 끝까지 들어야 돈을 돌려받나요?	HR-EDU-004 수료율 제외	HR-EDU-002 신청 기한	0.5331	틀림

"집에서 일해도"처럼 단어가 달라도 재택근무를 찾아오는 건 Semantic Search가 하는 일이다. 키워드 검색이면 못 찾았을 질문이다. 반대로 네 번째 질문은 실패했다. "돈을 돌려받나요"가 수료율 제외보다 신청 기한 쪽에 더 가깝게 붙은 것 같다. 정답 문서(HR-EDU-004)는 Top-2(0.4911)에 있었다. 의미 검색도 만능이 아니고, 표현이 빗나가면 가까운 다른 규정으로 새는 걸 봤다.

B. 유사도 점수 분포와 Threshold

FAISS는 관련 없는 질문에도 항상 Top-K를 준다. 그래서 "답이 있는 질문"과 "없는 질문"의 Top-1 점수를 나눠봤다.

구분	최소	평균	최대
Knowledge에 답이 있는 질문 6개	0.6913	0.7593	0.8096
Knowledge에 답이 없는 질문 3개	0.5273	0.5474	0.5647

이 미니 KB에서는 두 집단이 안 겹친다. 있는 질문의 최솟값(0.6913)이 없는 질문의 최댓값(0.5647)보다 높다. 중간값 0.63을 Threshold로 걸고, 그 아래는 Qwen을 호출하지 않고 "확인할 수 없습니다"로 처리해 봤다. 답이 있는 6개는 통과, 없는 3개는 전부 걸러졌다.

문서 6건짜리라서 숫자가 이렇게 예쁘게 갈린 것이다. 규정이 수백 건이면 점수가 겹칠 수 있다. 그래도 노트북이 "검색 결과가 있다고 정답은 아니다"라고 적어 둔 이유를, 점수 분포로 한번 확인해 본 의미가 있다.

C. top_k를 키우면 오히려 답이 흐려질 수 있다

70% 수료율 질문의 검색 결과만 top_k를 바꿔 봤다.

```
top_k=1  0.6913  HR-EDU-004  지원 제외 조건    ← 정답 문서만
top_k=2  0.6913  HR-EDU-004
```

```
          0.4828  HR-EDU-001  지원 한도 120만원  ← 여기부터 노이즈
top_k=3   + HR-EDU-002 신청 기한 (0.4801)
top_k=6   교육비 문서 전부 + 재택/휴가까지 딸려 들어옴
```

노트북 기본값이 top_k=2라서, 정답 문서 옆에 120만원 한도가 같이 Qwen으로 들어갔다. 모델이 "적게 지급될 것입니다"라고 지어낸 재료가 그 두 번째 문서였던 것 같다. K를 무조건 늘리는 게 좋은 게 아니다. 작은 모델일수록 관련 없는 Context를 무시하지 못한다.

D. LoRA에서 r이랑 target_modules를 바꿔 보기

워밍업은 q_proj, v_proj / r=8 한 가지다. 같은 Base 모델에 설정만 바꿔서 Trainable 수가 어떻게 변하는지 표로 뽑았다.

설정	r	target	Trainable	비율(%)	Adapter (FP32)
LoRA 적용 전	-	-	1,543,714,304	100.0000	-
워밍업 노트북 (q, v)	8	2	1,089,536	0.0705	4.16 MB
rank만 4로	4	2	544,768	0.0353	2.08 MB
rank만 16으로	16	2	2,179,072	0.1410	8.31 MB
rank만 32로	32	2	4,358,144	0.2815	16.62 MB
Attention 전체 (q,k,v,o)	8	4	2,179,072	0.1410	8.31 MB
train_lora.py 실제 설정	16	7	18,464,768	1.1820	70.44 MB

눈에 보이는 규칙.

- r을 2배로 올리면 Trainable도 거의 정확히 2배가 된다. A, B가 r에 비례하니까 당연하다.
- target을 q,v에서 Attention 전체로 넓혀도, 워밍업에서 r만 16으로 올린 것과 같은 1.09만 개가 아니라 217만 개가 된다. "어디에 붙일지"와 "얼마나 크게 만들지"가 서로 다른 손잡이다. 가이드3 Q12.
- 다음 실습 스크립트는 Attention + MLP까지 7개 모듈, r=16이다. 학습 대상이 약 1.18%로, 워밍업(0.07%)보다 훨씬 넓다. 그래도 전체의 1%대다. Full Fine-Tuning(100%)과는 비교가 안 된다.
- train_lora.py 설정에서 LoRA 텐서가 392개다. 7개 모듈 × A/B × 28층 = 392. 층마다 독립이라는 설명이 개수로도 맞는다.

E. 돌리다가 막힌 것

- Qwen은 Hugging Face 캐시(`~/.cache/huggingface/hub`)에 있어서 다시 받을 필요는 없었다. 스냅샷 경로까지 노트북이 찍어준다.

- 이 Mac에서 `faiss`를 import한 뒤에 BGE-M3 encode를 돌리면 프로세스가 죽는다 (exit 139). 노트북은 이미 돌아가 있는 환경에서 결과가 나왔고, 추가 실험은 IndexFlatIP와 같은 연산인 정규화 벡터 내적을 numpy로 재현했다. 문서 6건이면 전수 비교라 점수가 노트북과 동일했다 (0.7812, 0.8096 등).
- 16GB 메모리에서 Qwen 생성까지 한 스크립트에 묶으면 중간에 죽는 경우가 있었다. 생성 결과는 워밍업 노트북 출력을 쓰고, 검색 쪽 추가 실험만 따로 돌렸다.

오늘 정리하면 이렇다. 범용 모델은 회사 고유 Knowledge를 모른다. 그 지식을 검색으로 붙이면 당장 답은 나오지만, 작은 모델은 조건을 잘못 읽거나 옆 문서를 섞어 지어내기도 한다. LoRA는 그 습관을 고치기 위해 15억 개를 다시 학습하지 않고, 층 옆에 작은 파라미터만 얹는 방법이다. 다음 실습에서 그 작은 파라미터를 실제로 학습시키고, Base와 답이 얼마나 달라지는지를 본다.